

Debian 3D Design, Printing, and CNC Milling Setup Guide

This guide consolidates a robust, dependency-clean workflow for transforming [OpenSCAD](#) models into files ready for both 3D printing (additive) and CNC milling (subtractive) manufacturing on Debian Linux. It systematically bypasses broken or legacy package dependencies (`pycam` , `camotics` binary hooks) by utilising clean alternative toolchains.

NOTE: Do not use the `hull()` function in OpenSCAD on large geometric variations if processing performance is constrained.



FreeCAD vs. Dedicated Shop-Floor Workflows

[FreeCAD](#) is the standard, feature-complete open-source powerhouse for parametric 3D design and CAM (Computer-Aided Manufacturing) toolpath generation on Linux. Its dedicated *Path/CAM Workbench* and Python-scriptable backend are highly capable of handling both complex 3D surface milling and native CNC lathe operations.

While [FreeCAD](#) is a highly capable open-source powerhouse for parametric 3D design and CAM generation, its heavy graphical environment is not suited for the actual machining environment.

When you transition to physical production, you typically deploy dedicated, low-end hardware—such as an old shop laptop—to run your CNC mill or lathe. On the shop floor, running a bloated CAD package during a time-consuming machining operation is inefficient and risky. Instead, you need a minimalist, ultra-lightweight, and bulletproof pipeline.

This guide provides exactly that: a **low-overhead, command-line-first, or minimalist workflow** optimised for low-spec shop hardware, allowing you to prepare and execute files without the weight of FreeCAD.



Phase 1: CAD Engine & Model Translation (STEP & DXF)

OpenSCAD natively generates polygonal meshes. For high-fidelity engineering workflows, we use modern isolation tools to convert shapes to true B-Rep STEP files or 2D profiles.

1. Install System Prerequisites

```
sudo apt update
sudo apt install openscad curl wget
```

2. Install the **uv** Package Manager & Translate to STEP

Instead of wrestling with system paths, we use **uv** to handle sandbox translation routines cleanly:

```
# Download and install uv safely
curl -LSsf https://astral.sh/uv/install.sh -o /tmp/uv_install.sh
bash /tmp/uv_install.sh
source ~/.bashrc

# Permanently install the translation engine
uv tool install scad2step

# Run translation (Outputs: LetterBlock.step)
scad2step LetterBlock.scad

# Or,
scad2step yourOpenSCADmodel.scad
# (Outputs: yourOpenSCADmodel.step)
```

3. Generate 2D Profile Vector Data (For Flat CNC Profiles)

If your model features flat cutouts or pockets, project it to a 2D plane inside OpenSCAD rather than utilising a heavy 3D mesh:

```
openscad -o LetterBlock_2D.dxf -D "projection(cut = false) use_your_module();"
LetterBlock.scad
```



Phase 2: Lightweight Local 3D Inspection

Verify your generated 3D STEP files headlessly or visually without downloading bulk CAD suites.

1. Configure **Mayo** ApplImage

Download from: <https://github.com/fougue/mayo.git>

Download the C++ OpenCASCADE-backed viewer, mark it executable, and move it to a dedicated directory:

```
mkdir -p ~/PortablePrograms
# Place your downloaded 'Mayo-0.10.0-x86_64.AppImage' into this directory
chmod +x ~/PortablePrograms/Mayo-0.10.0-x86_64.AppImage
```

2. Create a Permanent System Alias

Append an explicit alias to your terminal environment profile to launch Mayo with simple strings:

```
echo "alias mayo='~/PortablePrograms/Mayo-0.10.0-x86_64.AppImage'" >> ~/.bashrc
source ~/.bashrc
```

Usage: `mayo LetterBlock.step &`

3. Create a Desktop Entry

```
touch ~/.local/share/applications/Mayo-STEP-Viewer.desktop
chmod +x ~/.local/share/applications/Mayo-STEP-Viewer.desktop
nano ~/.local/share/applications/Mayo-STEP-Viewer.desktop
```

Paste the following lines into the file.

```
[Desktop Entry]
Type=Application
Name=Mayo AppImage
Comment=3D STEP file viewer
Icon=choice-rhomb
Exec=/mnt/hdd2/PortablePrograms/Mayo-0.10.0-x86_64.AppImage
Terminal=false
Categories=;
```



Phase 3: Additive Manufacturing (3D Printing G-code)

PrusaSlicer provides a mature command-line implementation to gather mesh geometry information and slice components.

1. Install Slicer Bundle (318 MB)

```
sudo apt install prusa-slicer
```

2. Terminal Analysis & G-code Export

```
# Query technical boundaries of your STEP model
prusa-slicer --info LetterBlock.step

# Export standard additive layered toolpaths
prusa-slicer --export-gcode LetterBlock.step
```



Phase 4: Subtractive Manufacturing (CNC Milling G-code)

CNC routing requires manual inputs for paths, tooling, and feed metrics. We split this work based on design attributes.

1. 2.5D Profiling, Pocketing, and Engraving (Flat Workspace)

For shapes featuring absolute horizontal layers with vertical walls, route vectors through `dxfg2gcode` :

```
sudo apt install dxfg2gcode
dxfg2gcode LetterBlock_2D.dxf
```

- **Stepdown Logic:** Inside the GUI (yes, it is a GUI application which also has a TUI), map target layers under your tool offset rules, set `axis3_mill_depth` (total depth), define individual cut slices (`axis3_slice_depth`), and select **Export** → **Optimize and Export Shapes**.

2. Complex 3D Surfacing & Native CAM Foundations

If your part features non-flat 3D topographies, utilise Debian-native industrial CAM utilities to generate paths securely without Python interpreter fragmentation:

The LinuxCNC stack (687 MB):

NOTE: * Do not install the LinuxCNC stack if you do not want to use it. It is heavy. Keep it only if configuring tool tables or native conversational loops for a Lathe.

```
sudo apt install linuxcnc-ospace
```

This deploys the robust LinuxCNC utility architecture (`image-to-gcode` interpreters and machine solvers) natively inside your environment.

Verifying Final Pipeline Tool Availability

Ensure your environment registers each utility without library error blocks by executing:

```
openscad --version
scad2step --help
prusa-slicer --version
dxf2gcode --help
```

This gives you an immediate, low-overhead pipeline architecture completely managed locally on Debian.

If you still want to install [CAMotics](https://camotics.org/) for some reason.

<https://camotics.org/> Download the Debian package `camotics_1.2.0_amd64.deb` . *The version number may be different in the future.*

Go to the directory where you downloaded the CAMotics package.

```
sudo apt-get update
sudo apt-get install -y gdebi
sudo gdebi camotics_1.2.0_amd64.deb
```

Also install `libqt5websockets5-dev`

```
sudo apt-get update
sudo apt-get install -y libqt5websockets5-dev
```

For a missing libv8:

```
wget http://ubuntu.com -O /tmp/libv8.deb
```

```
find /usr/lib -name "libv8.so*"
```

Copy the path.

And,

```
sudo ln -s /usr/lib/x86_64-linux-gnu/libv8.so /usr/lib/libv8.so.3.14.5
```

Download the exact library from Ubuntu (yes, even though it goes **against the rules of using Debian**):

```
wget http://mirrors.kernel.org/ubuntu/pool/universe/libv/libv8-3.14/libv8-3.14.5_3.14.5.8-5ubuntu2_amd64.deb
```

```
sudo dpkg -i libv8-3.14.5_3.14.5.8-5ubuntu2_amd64.deb
```

This will fix the missing libv8 library warning.

There's another utility called [Open STEP Viewer](https://openstepviewer.com/). However, you need to register an account before downloading it.

<https://openstepviewer.com/>

Installing CAMotics Legacy Package (Hybrid Fix) [Alternative Method]

If you explicitly require CAMotics toolpath simulation on modern Debian, use the exact file-mapping and package sequence required to satisfy its compiler tracking:

```
# 1. Install prerequisites and Qt5 frameworks
sudo apt update
sudo apt install -y gdebi libqt5websockets5 libqt5opengl5 libnode-dev

# 2. Fetch and deploy the historical libv8 runtime package
wget http://kernel.org -O /tmp/libv8.deb
sudo dpkg -i /tmp/libv8.deb

wget http://mirrors.kernel.org/ubuntu/pool/universe/libv/libv8-3.14/libv8-3.14.5_3.14.5.8-5ubuntu2_amd64.deb

sudo dpkg -i libv8-3.14.5_3.14.5.8-5ubuntu2_amd64.deb

# 3. Apply the fallback symlink mapping to satisfy strict runtime lookups
sudo ln -sf /usr/lib/x86_64-linux-gnu/libv8.so /usr/lib/libv8.so.3.14.5

# 4. Install the application package
sudo gdebi camotics_1.2.0_amd64.deb
```

(Note: If toolpath generation continues to return C++ symbol errors on your project file, look to native options like `dx2gcode` instead).

Before you dive headfirst into your machine shop workflows, keep these two critical structural differences in mind regarding the software stack you just put together:

1. CNC Milling (Ready to Go)

Your current pipeline is fully optimised for your mill. You can immediately feed your extruded 2D layouts into `dx2gcode`, set your cut slices (`axis3_slice_depth`), and generate your `.ngc` toolpaths natively on your Debian system.

2. CNC Lathing (Important Caveat)

While `dx2gcode` is incredible for multi-axis milling, it does not support turning profiles (lathe toolpaths). For a lathe, the machine expects specialised **G-code cycles** (like `G71` or `G72` roughing and finishing paths) that track your tool along a rotational centerline rather than cutting a flat 2D plane or pocket matrix.

Since you are using OpenSCAD, the absolute cleanest way to program your lathe profiles without bloating your Debian system is to export your *rotational part cross-section as a 2D DXF line profile*, and run it through a lightweight, web-based lathe CAM tool like **TurnCAD** or use FreeCAD's **Path/CAM Lathe Workbench** headlessly.

If you intend to operate a lathe or manage a dedicated machine control centre, keeping `linuxcnc-ospace` is the most logical choice.

While the suite requires a chunk of storage (687 MB), it does something no other lightweight package on Linux can do: it serves as both your **G-code generator** for **rotational cuts** and your **real-time machine controller**.

By deciding to keep it, you gain massive advantages for your lathe setup:

1. Built-In Lathe G-code Wizards (No CAM Required)

LinuxCNC includes native, built-in conversational turning wizards (like Lathe Macro Cycles and custom Python wizard wrappers) directly inside its AXIS graphical interface. Instead of dealing with an external CAM program to design a simple turned pin or bush, you can input variables directly into the LinuxCNC interface to dynamically compile roughing, facing, and threading cycles (G71, G72, G76).

2. Native Centerline Path Preview

Standard *milling visualisers* (like `dxfg2gcode` or `mayo`) read geometry *inside an X/Y/Z Cartesian matrix*. A lathe tracks coordinates on a *specialised Radial/Centerline axis system* (X for diameter, Z for length). The AXIS frontend inside `linuxcnc-ospace` *instantly flips to a native 2D lathe layout*, correctly *mapping tool offsets* and *radius compensation* for turning inserts.

Adjusting Your Final Workflow

Since you are keeping LinuxCNC, your complete manufacturing pipeline becomes beautifully simple:

-
- **3D Printing:** `openscad` → `prusa-slicer` (Additive G-code)
- **CNC Milling (2.5D):** `openscad` → DXF export → `dxfg2gcode` (Milling G-code)
- **CNC Lathing & Live Machine Control:** `openscad` (**2D layout cross-section**) → **LinuxCNC Interactive Console** (*Turning G-code execution* & live tracking)
- You are now armed with a rock-solid, production-grade Linux workstation.

[1] <https://www.linuxcnc.org>

[2] <https://linuxcnc.org>

[3] <https://forum.linuxcnc.org>

[4] <https://linuxcnc.org>

[5] <https://github.com>

[6] <https://www.youtube.com>

Attribution & Collaboration

This document was created through a direct conversation between **Pinaki Sekhar Gupta** and **Google Gemini/Search AI** (as the Technical Documentation Assistant).

- **System & Workflow Architecture:** Directed by Pinaki Sekhar Gupta, establishing the specific constraints for low-overhead shop-floor hardware, Debian system configuration choices, and direct software testing.
 - **Technical Synthesis & Documentation Compilation:** Written and formatted by Google Search AI, parsing dependencies and generating functional scripts based on real-time execution feedback.
-